

How Aroha works, end to end

A plain-English map of the product: what happens from the moment someone opens the app to the moment they get their reading, and exactly which technology does each job.

Traced directly from the live codebase across the backend, web app, mobile shell, and landing site · 12 Jul 2026

§0 The short version

Aroha is **one backend** serving **three separate front doors**: a web app, a thin mobile wrapper around that same web app, and an unrelated marketing site.

All the actual astrology — charts, dashas, panchang, matching — is **calculated in code**, not by AI. It's exact and reproducible every time.

AI is only used to **write the words** around those calculations — the daily horoscope narration, chat replies, muhurta write-ups — and it's always fed the pre-computed facts first, so it narrates rather than invents.

§1

The flow, step by step

Read top to bottom — each row is one stage a request passes through, in order.

01 • Where the user starts

Web app (PWA)

The main product — used directly in a phone or desktop browser.

Next.js 15

React 19

Tailwind CSS

Mobile app

Not a separate build — a native wrapper that just opens the web app in a WebView, plus push notifications and the back button.

Capacitor

Android / iOS

Landing site

A separate, purely public marketing site — no account or data flows through it.

Next.js 15

Three.js



02 • Sign in

Firebase Authentication — phone OTP

Verified entirely on the device. The backend never sees the raw phone number — only a signed identity token.

Firebase Auth

Phone OTP



03 • One backend for everything

Backend API

Every screen — Kundli, Panchang, Horoscope, Chat, Compatibility — calls this one API. It re-checks the identity token and loads (or creates) the matching account before doing anything else.

Hono + TypeScript

Runs on one EC2 server



04 • Two engines, doing very different jobs

Astro Engine — deterministic

Planet positions, houses, dashas, doshas, yogas, panchang and matchmaking are plain math against real ephemeris data. No AI touches this — the numbers are exact and identical every time they're run.

Swiss Ephemeris

TypeScript

Intelligence layer — grounded AI

For anything that has to read like a paragraph — the horoscope write-up, AI chat, timing analysis — the Astro Engine's facts are handed to a language model with instructions to narrate them, not invent new ones.

Groq

→ OpenRouter

→ NVIDIA NIM



05 • Everything that needs to be remembered

Data store

Accounts, saved birth charts, generated kundlis, credit balances and preferences all live in one database. A separate cache layer exists for speed but is only lightly used today.

PostgreSQL (Supabase)

Drizzle ORM

Redis



06 • What fires off in the background

Push notification

When a new daily horoscope is ready, an alert is pushed to the device.

Firebase Cloud Messaging

Billing

Credit packs, coupons and checkout exist end-to-end in the code.

mocked – no live gateway yet

Ops alerting

A bot is wired up to flag backend issues to the team.

Telegram Bot API



07 • Back to the user

Rendered, in their language

The response is rendered in the relevant screen and shown in whichever of the 7 supported languages the user has chosen.

i18next

English · Hindi · Bengali · Marathi · Telugu · Tamil · Gujarati

§2 Backend structure

The API is one TypeScript codebase (`src/`). Every feature module follows the same four-file shape, so once you've read one, you've read the pattern for all sixteen.

```
src/
├─ config/           environment & service setup – Supabase, Redis, env vars
├─ db/
│   └─ migrations/   16 versioned SQL migrations (Drizzle Kit)
│   └─ schema.ts      the single source of truth for every table – see §3
├─ lib/
│   └─ astro-engine/  the deterministic chart-math engine: calculations, charts,
│   │                 dashas, doshas, yogas, muhurta, matching, numerology, panchang
│   └─ astro-tools/   higher-level readings built on the engine: daily synthesis,
│   │                 dasha narration, transits, tara-bala
│   └─ llm/           groq-client · openrouter-client · nim-client, plus the
│   │                 dispatcher that falls through them in order
│   └─ swarm/         the AI chat / onboarding / matchmaking agent pipeline:
│   │                 gateway → metrologist → [synthesizer, profiler, scholar] → aggregator
│   └─ notifications/ fcm.ts (push) · telegram.ts (ops alerts)
│   └─ cache/         Redis helpers
│   └─ usage/         AI token / cost logging
│   └─ shared/        cross-module utilities
├─ middleware/       auth.ts verifies the Firebase token on every request
├─ modules/          one folder per feature – each has routes · service · repo · schemas
│   └─ auth/ users/ birth-profiles/ kundli/ horoscope/
│   └─ astro/ purchase-plan/ billing/ device-tokens/
│   └─ preferences/ feedback/ legal/
│   └─ health/ health-report/
│   └─ cron/ telegram-bot/
├─ scripts/          one-off maintenance scripts
├─ types/            shared TypeScript types
├─ app.ts            wires every module's routes onto the server
└─ index.ts          process entry point
```

***.routes.ts** defines the HTTP endpoints and request/response shape for that module

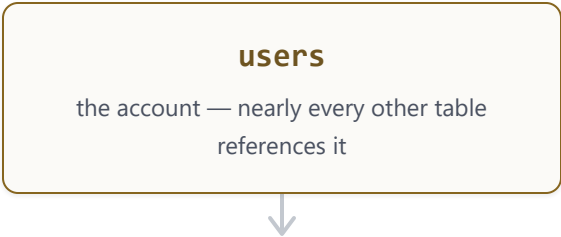
***.service.ts** the business logic — what actually happens when a route is called

***.repo.ts** the only layer allowed to talk to the database for that module

***.schemas.ts** Zod validation, which doubles as the OpenAPI docs at `/docs`

§3 Database schema

One PostgreSQL database, 17 tables. Almost everything hangs off a single **users** row — that's the hub.



PROFILE & CONSENT

- **birth_profiles** — saved charts for other people (family, matches)
- **device_push_tokens** — devices registered for push
- **user_consent_log** — append-only consent audit trail

ASTROLOGY DATA

- **kundlis** — one precomputed natal chart per user
- **daily_horoscopes** — one row per user per period
- **house_insights** — one reading per user per house (1–12)
- **purchase_plans** — timing analysis, links to a kundli
- **panchang_cache** — *shared by date + location, no user link*

BILLING & CREDITS

- **credit_transactions** — the credit-balance ledger
- **orders** — credit-pack purchases
- **user_subscriptions** — links a user to a plan
- **coupons, subscription_plans** — *standalone catalogs*

ENGAGEMENT & OPS

- **prediction_feedback** — ratings on predictions
- **ai_usage** — LLM token / cost per call
- **precompute_jobs** — background generation job tracking

TABLE REFERENCE

users	The account itself: identity, the birth event (DOB/time/place), chart-calculation preferences (ayanamsa, house system, dasha system), locale, consent timestamps, onboarding state, credit balance. ~50 columns — by far the widest table.
birth_profiles	<code>owner_user_id</code> → users · relationship (partner / child / parent / prospective match / ...) · its own birth-event fields · an explicit consent flag for storing someone else's data
device_push_tokens	<code>user_id</code> → users · token · platform (ios / android / web) · revoked_at
user_consent_log	<code>user_id</code> → users (delete-restricted, so the audit trail survives account deletion) · consent_type · granted / withdrawn — append-only

kundlis	<code>user_id</code> → <code>users</code> , one-to-one · status (pending / generating / ready / failed) · chart, dasha, yoga, dosha and ashtakavarga data, each its own JSON column
daily_horoscopes	<code>user_id</code> → <code>users</code> · period (daily / tomorrow / weekly / monthly / yearly) + a period key · structured reading (JSON) · status
house_insights	<code>user_id</code> → <code>users</code> · house (1–12) · one row per (user, house) pair
purchase_plans	<code>user_id</code> → <code>users</code> , <code>chart_id</code> → <code>kundlis</code> · category (vehicle / home / commercial) · resolved booking & delivery dates · analysis (JSON)
panchang_cache	keyed by date + reference point (unique) · lat/lon · data (JSON) — shared across every user, no foreign key at all
subscription_plans	standalone plan catalog — name, monthly price, feature flags
user_subscriptions	<code>user_id</code> → <code>users</code> , <code>plan_id</code> → <code>subscription_plans</code> · status (active / cancelled / expired / trial)
credit_transactions	<code>user_id</code> → <code>users</code> · delta · reason · balance_after — an append-only ledger, not just a running total
coupons	standalone — code (unique) · discount type (percent / flat) · redemption limits
orders	<code>user_id</code> → <code>users</code> , <code>coupon_id</code> → <code>coupons</code> · amount · <code>gateway_provider</code> defaults to 'mock' — see §6
prediction_feedback	<code>user_id</code> → <code>users</code> · rating · helpful (boolean) · free-text comment
ai_usage	<code>user_id</code> → <code>users</code> (nullable) · agent · model · tokens in/out · duration — one row per LLM call
precompute_jobs	<code>user_id</code> → <code>users</code> · period_type / period_key · status — tracks background horoscope/insight generation

Walking through it in words

01 Someone opens Aroha

Whether they installed the mobile app or opened the site in a browser, they land on the same Next.js web app — the mobile "app" is just that same site loaded inside a native wrapper (**Capacitor**), which is also what lets it send push notifications and handle the phone's back button. The separate landing site only exists to funnel new people toward downloading the app; it has no account logic of its own.

02 They sign in with their phone number

Firebase Authentication handles the OTP entirely on the device. Once verified, the app exchanges Firebase's identity token for an app session by calling **POST /v1/auth/session** — the backend never has to store or process the raw phone number itself.

03 Every screen talks to one backend

Kundli, Panchang, Horoscope, AI Chat and Compatibility are all just different calls into the same TypeScript API. That token gets re-verified on every request before anything happens.

04 The numbers come first, always

Requesting a kundli or panchang triggers the in-house Astro Engine, which runs real astronomical calculations (Swiss Ephemeris) for planetary positions, houses, dashas, doshas and yogas. This step never involves AI — it's deterministic, so the same birth details always produce the same chart.

05 AI only shows up to narrate

For the daily horoscope write-up, AI chat replies, and the "auspicious timing" analysis, the backend hands the Astro Engine's output to a language model and asks it to turn facts into readable prose. It tries Groq first; if that's unavailable it automatically falls back to OpenRouter, then to NVIDIA NIM — the user never sees the difference.

06 Everything durable lands in Postgres

Accounts, saved birth profiles, generated charts, credit balances and preferences are all written to one PostgreSQL database (hosted on Supabase, accessed through Drizzle ORM). A Redis cache is available for speed but most paths don't lean on it yet.

07 A few things happen in the background

A freshly generated daily horoscope triggers a push notification through Firebase Cloud Messaging. Credit purchases run through a fully-built billing flow — packs, coupons, checkout — except the actual payment

gateway isn't connected yet, so today it's a mocked transaction. A Telegram bot is wired up to alert the team if the backend runs into trouble.

08 The answer comes back, in the user's language

The result renders in whichever screen asked for it, translated into the user's chosen language out of the 7 currently supported (English, Hindi, Bengali, Marathi, Telugu, Tamil, Gujarati) via i18next.

§5 The stack, by layer

CLIENT

Web app	Next.js 15, React 19, TypeScript, Tailwind CSS, Framer Motion — the main product surface, used both in-browser and inside the mobile wrapper
Mobile shell	Capacitor (Android / iOS) — wraps the web app for app-store distribution, adds native push notifications and back-button handling
Landing site	Next.js 15, Three.js / React Three Fiber, Framer Motion — separate codebase, marketing only
Sign-in	Firebase Authentication, phone OTP

BACKEND

API framework	Hono + TypeScript, with auto-generated OpenAPI docs
Database access	Drizzle ORM
Auth verification	Firebase Admin SDK
Astrology math	In-house Astro Engine on swisseph-wasm (Swiss Ephemeris) — no external astrology API

AI

Narration models	Groq (primary) → OpenRouter (fallback) → NVIDIA NIM (final fallback), automatic on failure
Used for	Daily horoscope narration, AI chat, purchase-plan / muhurta analysis — always grounded in Astro Engine output, never freeform

DATA

Database	PostgreSQL, hosted on Supabase
Cache	Redis — configured, lightly used today

NOTIFICATIONS & OPS

Push	Firebase Cloud Messaging
Team alerts	Telegram Bot API

INFRASTRUCTURE

Backend hosting	A single AWS EC2 instance (Mumbai), process-managed with pm2, deployed by hand — no CI/CD pipeline yet
Web / landing hosting	Standard Next.js hosting, deployed on push

§6

Good to know

- **Billing isn't live yet** — the whole credit-purchase flow is built, but it's connected to a mock, not a real payment gateway.
- **AI never invents astrology** — it only writes prose around numbers the Astro Engine already computed.
- **One server runs the whole backend**, and deploys are a manual process today rather than an automated pipeline.
- **The mobile app isn't a separate codebase to maintain** — since it's a wrapper around the web app, most product changes go live the moment the web app deploys, with no app-store review needed.
- **Full language support** — all 7 languages are wired through both the interface and the AI-generated content, not just static text.

Reflects the codebase as of 12 Jul 2026 — worth re-checking after major backend or app changes, since this is a snapshot, not a live-synced document.